

LECTURE 11

Scheduling Algorithms: SJF, SRTF, and Round Robin

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Shortest-Job-First (SJF): why it's provably optimal, and the catch that keeps it from being used exactly as described
- Shortest-Remaining-Time-First (SRTF): SJF's preemptive version, with a worked example showing a preemption actually happen
- Round Robin: fair time-slicing, worked through multiple rounds, and how the time quantum's size changes everything
- Priority scheduling, the starvation problem it creates, and aging as the fix
- How priority scheduling and Round Robin compare — and why real schedulers usually need both

Shortest-Job-First (SJF)

SJF schedules whichever ready process has the shortest next CPU burst. Like FCFS, it is normally non-preemptive: once a process starts running, it keeps the CPU until it finishes — but every time the CPU becomes free, the scheduler picks the shortest burst among whoever is ready at that moment.

SJF is provably optimal for minimizing average waiting time

Among all non-preemptive algorithms, for a fixed, known set of CPU burst lengths, SJF gives the minimum possible average waiting time. Running a short job before a long one shortens the wait of everything behind the long job by the long job's entire length, while only lengthening the long job's own wait by the short job's length.

Among all non-preemptive scheduling algorithms, SJF's average waiting time cannot be beaten for a fixed, known set of burst lengths.

The Catch: SJF Needs the Future

SJF needs to know each process's next CPU burst length before running it — and in general, the OS cannot know that in advance. The practical fix is to estimate the next burst from the lengths of a process's previous bursts, most commonly using exponential averaging:

$$\tau(n+1) = \alpha \cdot t(n) + (1 - \alpha) \cdot \tau(n)$$

where $t(n)$ is the process's actual, just-measured n -th burst length, $\tau(n)$ was the previous estimate, and α (between 0 and 1) controls how heavily the most recent burst is weighted against the accumulated history.

In practice, "SJF" almost always means "scheduling by estimated next burst," not by some impossible perfect knowledge of the future.

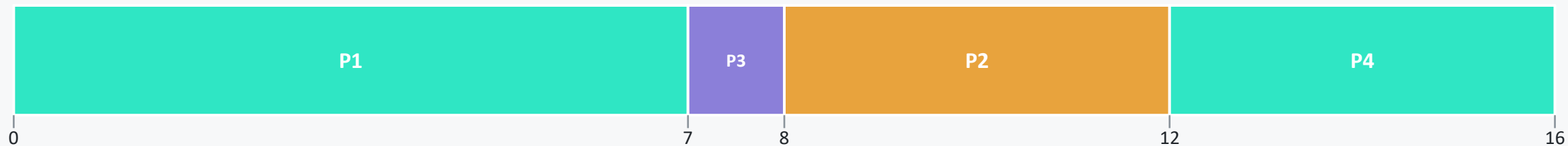
Worked Example: Non-Preemptive SJF

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

At $t=0$, only P1 has arrived — the scheduler runs it for its full burst of 7, finishing at $t=7$. By $t=7$, P2, P3, and P4 have all arrived; shortest is P3 (burst 1), so it runs next. At $t=8$, P2 and P4 are tied at burst 4 — ties break by earlier arrival, so P2 (arrived $t=2$) runs before P4 (arrived $t=5$).

Gantt Chart: Non-Preemptive SJF

Shortest-burst-first among ready processes at each decision point



Run order: P1 (0→7), P3 (7→8), P2 (8→12), P4 (12→16).

SJF: Waiting and Turnaround Times

Process	Arrival	Burst	Start	Completion	Waiting	Turnaround
P1	0	7	0	7	0	7
P3	4	1	7	8	$7-4=3$	$8-4=4$
P2	2	4	8	12	$8-2=6$	$12-2=10$
P4	5	4	12	16	$12-5=7$	$16-5=11$

Average waiting time = $(0 + 3 + 6 + 7) / 4 = 16 / 4 = 4$ Average turnaround time = $(7 + 4 + 10 + 11) / 4 = 32 / 4 = 8$

An average waiting time of 4 is exactly the payoff SJF's optimality promises — but P4 still waits 7 units purely because it tied with P2 and lost the tiebreak; even an optimal average can still treat one specific process worse than another.

Shortest-Remaining-Time-First (SRTF)

SRTF is SJF's preemptive cousin: instead of committing to a process once it starts, the scheduler re-evaluates every time a new process arrives.

If the new arrival's burst is shorter than the remaining time of whichever process is currently running, the running process is preempted immediately and the new, shorter process takes over.

SRTF is what "SJF" means whenever preemption is allowed — the non-preemptive version only ever had the chance to choose shortest-first at the moments the CPU happened to be free already; SRTF can act on that information the instant it becomes available.

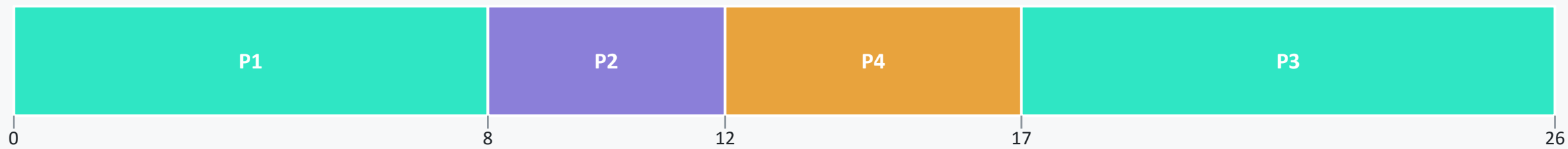
Comparing SJF and SRTF on the Same Workload

To see preemption actually happen, run a new process set through both non-preemptive SJF and SRTF side by side.

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Non-Preemptive SJF (No Preemption Possible)

P1 runs to completion regardless of what arrives later

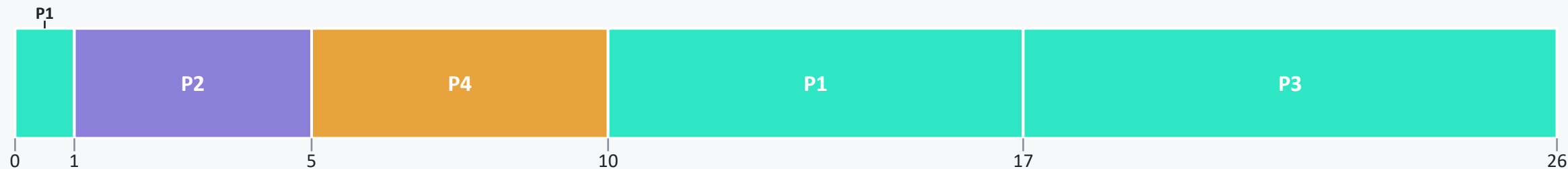


Process	Arrival	Burst	Start	Completion	Waiting	Turnaround
P1	0	8	0	8	0	8
P2	1	4	8	12	7	11
P4	3	5	12	17	9	14
P3	2	9	17	26	15	24

Average waiting time (SJF) = $(0 + 7 + 9 + 15) / 4 = 31 / 4 = 7.75$

SRTF: P1 Is Preempted by P2 at t=1

At t=1, P2's burst (4) < P1's remaining time (8-1=7) — P2 preempts



Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	8	17	17	17-8=9
P2	1	4	5	4	4-4=0
P4	3	5	10	7	7-5=2
P3	2	9	26	24	24-9=15

Average waiting time (SRTF) = $(9 + 0 + 2 + 15) / 4 = 26 / 4 = 6.5$

SRTF Beats SJF on the Identical Workload

Non-Preemptive SJF

Average waiting time = 7.75

P1 runs to completion once started — P2's short burst waits behind P1's full remaining time.

SRTF

Average waiting time = 6.5

The preemption at $t=1$ lets P2 finish almost immediately instead of waiting behind P1's remaining 7 units.

SRTF's average waiting time is always less than or equal to non-preemptive SJF's, for the same arrival/burst data, because SRTF can always act on information the moment it arrives, while non-preemptive SJF can only act on it the next time the CPU happens to be free.

Round Robin (RR)

Round Robin abandons "shortest first" entirely in favor of fairness: every process in the ready queue gets a fixed-length turn, called a time quantum (commonly 10–100 ms in real systems).

If a process hasn't finished by the end of its quantum, it's preempted and sent to the back of the ready queue to wait for its next turn.

No process can be skipped indefinitely, no matter how long other processes' bursts are — the opposite failure mode from FCFS's convoy effect.

Worked Example: Round Robin, Quantum = 4

Three processes, all arriving at $t=0$. Each process runs for $\min(\text{remaining}, 4)$, then — if anything remains — goes to the back of the queue.

Process	Burst Time
P1	10
P2	5
P3	8

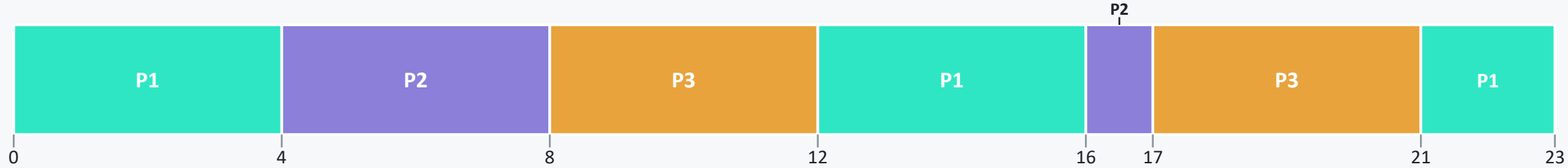
Ready queue starts as [P1, P2, P3].

Tracing Round Robin, Round by Round

- $t=0$: run P1 for $\min(10,4)=4$. Remainder 6. Queue: [P2, P3, P1].
- $t=4$: run P2 for $\min(5,4)=4$. Remainder 1. Queue: [P3, P1, P2].
- $t=8$: run P3 for $\min(8,4)=4$. Remainder 4. Queue: [P1, P2, P3].
- $t=12$: run P1 for $\min(6,4)=4$. Remainder 2. Queue: [P2, P3, P1].
- $t=16$: run P2 for $\min(1,4)=1$. Remainder 0 — P2 completes at $t=17$.
- $t=17$: run P3 for $\min(4,4)=4$. Remainder 0 — P3 completes at $t=21$.
- $t=21$: run P1 for $\min(2,4)=2$. Remainder 0 — P1 completes at $t=23$.

Gantt Chart: Round Robin, Three Rounds

Quantum = 4; a process returns to the back of the queue if work remains



Every process got CPU time within the first 12 units despite P1’s long 10-unit total burst — exactly the fairness guarantee FCFS couldn’t offer.

Round Robin: Waiting and Turnaround Times

Process	Burst	Completion	Turnaround	Waiting
P1	10	23	23	$23 - 10 = 13$
P2	5	17	17	$17 - 5 = 12$
P3	8	21	21	$21 - 8 = 13$

Average waiting time = $(13 + 12 + 13) / 3 = 38 / 3 \approx 12.67$

Average turnaround time = $(23 + 17 + 21) / 3 = 61 / 3 \approx 20.33$

Choosing the Time Quantum

Too Large

Round Robin degenerates toward FCFS — if the quantum exceeds every process's burst length, each process finishes in a single turn and the convoy effect reappears.

Too Small

The overhead of constant context switching (dispatch latency, paid on every preemption) starts to dominate — the CPU spends more time switching than running.

A practical rule of thumb

A well-chosen quantum should be large enough that roughly 80% of CPU bursts are shorter than it — most processes then finish in a single turn (behaving almost like SJF for the common case), while the rare long process is still kept from monopolizing the CPU for more than one quantum at a time.

Priority Scheduling

Priority scheduling assigns every process a priority number, and always runs the highest-priority ready process next — by convention, a smaller number means a higher priority.

SJF is, in fact, a special case of priority scheduling where the priority is simply the (estimated) next CPU burst length — shorter burst, higher priority.

Worked Example: Priority Scheduling

Four processes, all arriving at $t=0$ (lower number = higher priority).

Process	Burst Time	Priority
P1	4	3
P2	3	1
P3	2	4
P4	1	2

Run order follows priority directly: P2 (1), then P4 (2), then P1 (3), then P3 (4).

Gantt Chart: Priority Scheduling

Lower number = higher priority



Process	Priority	Start	Completion	Waiting Time
P2	1	0	3	0
P4	2	3	4	3
P1	3	4	8	4
P3	4	8	10	8

Average waiting time = $(0 + 3 + 4 + 8) / 4 = 15 / 4 = 3.75$

Starvation

P3 — last priority — waits 8 full time units despite having been ready since $t=0$, purely because three higher-priority processes kept cutting ahead of it.

A low-priority process can wait forever

Now imagine a steady stream of new high-priority processes continuing to arrive: P3 could in principle wait forever, never once being the highest-priority ready process at the moment the CPU frees up. This is starvation — a direct consequence of priority scheduling having no built-in guarantee that a low-priority process's wait is ever bounded, unlike FCFS or Round Robin.

Aging: The Fix for Starvation

Aging is the standard fix: periodically increase the priority of every process that has been waiting, the longer it waits.

Eventually, even P3's priority climbs high enough that it becomes the highest-priority ready process and finally runs — aging guarantees that every process's effective priority eventually catches up, turning an unbounded wait into a bounded one.

Priority Scheduling vs. Round Robin

	Priority Scheduling	Round Robin
Respects importance	Yes — an urgent process runs first	No — every process is treated identically
Starvation risk	Yes, without aging	No — the queue always moves forward
Fairness	No	Yes
Needs extra machinery to be safe	Aging	None

Most real operating systems don't choose one or the other; they combine both ideas — priority between groups of processes, Round Robin fairness within a group — exactly the multilevel queue structure Lecture 12 introduces next.

Key Takeaways

- SJF provably minimizes average waiting time among non-preemptive algorithms, but needs burst lengths it can only estimate, typically via exponential averaging
- SRTF, SJF's preemptive version, can act on a shorter arrival immediately — preempting P1 for P2 at $t=1$ dropped average waiting time from 7.75 (SJF) to 6.5 (SRTF) on the identical workload
- Round Robin guarantees fairness through a fixed time quantum; too large a quantum degenerates toward FCFS, too small lets context-switch overhead dominate — the 80%-of-bursts rule balances the two
- Priority scheduling can starve low-priority processes indefinitely; aging guarantees every process's wait is eventually bounded
- Priority scheduling respects importance but risks starvation; Round Robin is fair but blind to importance — real schedulers combine both, which is where Lecture 12 picks up